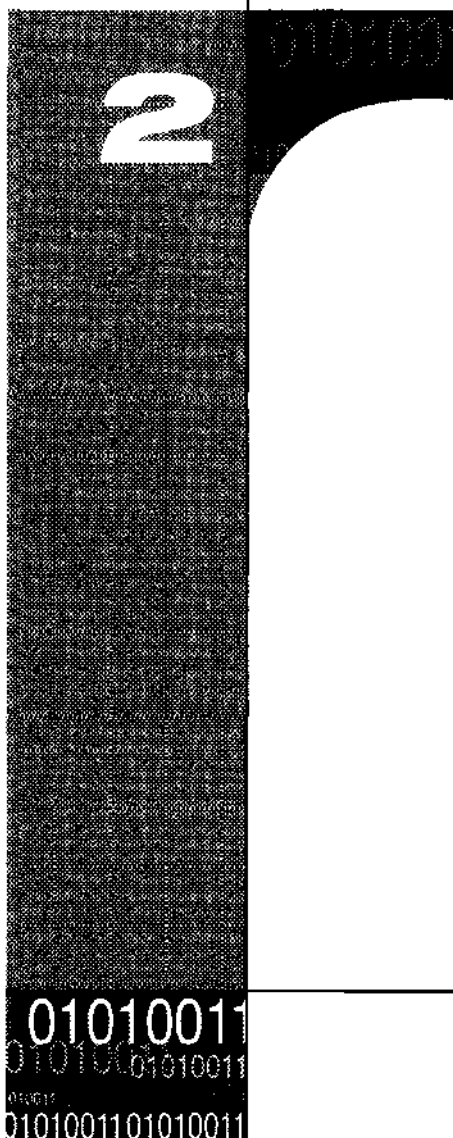
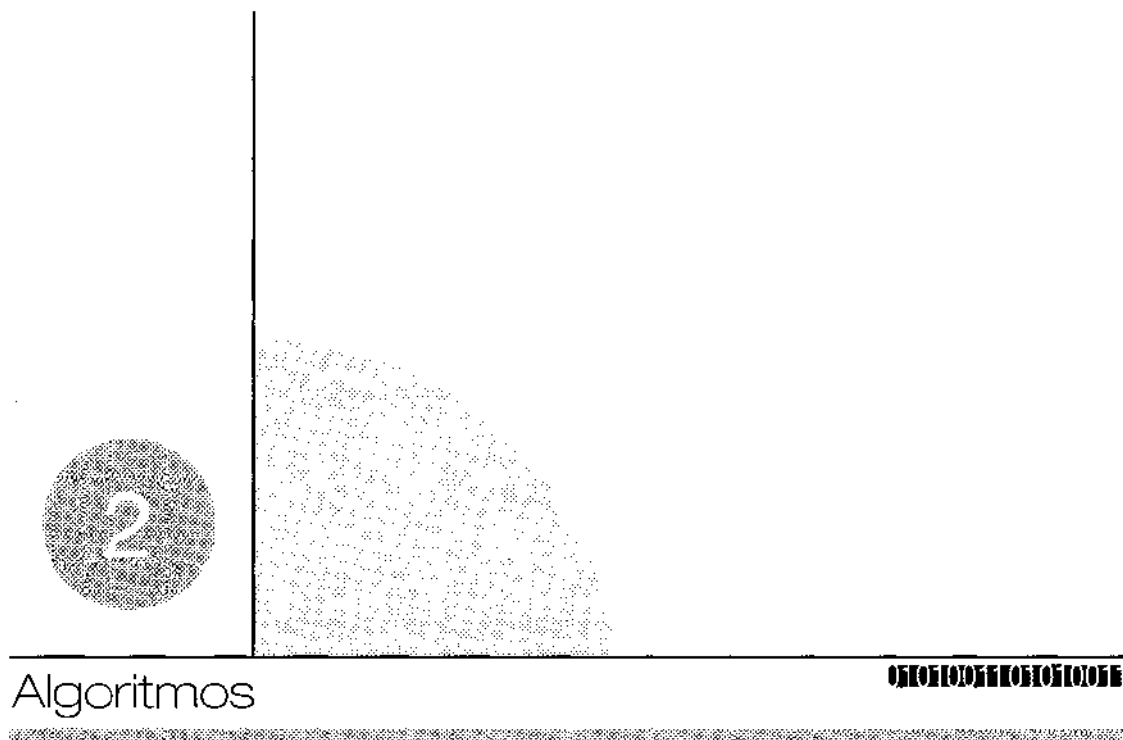




Capítulo 2

Algoritmos



Objetivo

En este capítulo se parte de la definición de algoritmo como especificación ordenada de la solución a un problema del mundo real.

El eje temático está centrado en las estructuras de control que permitirán expresar la secuencia de decisiones y de órdenes a ejecutar por una computadora en la solución de un problema.

Asimismo, se introducen los conceptos de modularización o descomposición de un problema como un mecanismo de reducción de la complejidad y simplificación de la tarea a resolver; de documentación de algoritmos como un requerimiento para permitir soluciones que se puedan comprender y mantener en el tiempo; de eficiencia como una medida de la *performance* del algoritmo elegido y de corrección de los algoritmos en la búsqueda de soluciones que respondan a los requerimientos del mundo real.

Por último, partiendo del concepto de datos simples y de variables vistos en el primer capítulo, se trabaja sobre la noción de localidad de las variables en un programa o módulo de programa.

2.1 Estructuras de control

En el capítulo 1 se definió algoritmo como la especificación rigurosa de la secuencia de pasos (instrucciones) a realizar sobre un autómata para alcanzar un resultado deseado en un tiempo finito.

Las instrucciones representan las operaciones que ejecutará la computadora al implementar el algoritmo. Estas instrucciones utilizan cierta información en forma de constantes o variables, que se denominan datos.

Todos los lenguajes de programación tienen un conjunto mínimo de instrucciones que permiten especificar el control propio del algoritmo que se requiere implementar. Este conjunto mínimo debe contener estructuras para la asignación, decisión e iteración.

2.1.1 Estructuras de control básicas

2.1.1.1 Secuencia

La estructura de control más simple está representada por una sucesión de operaciones (por ej. asignaciones, donde asignar significa almacenar valores constantes o variables), en la que el orden de ejecución coincide con el orden físico de aparición de las instrucciones.

La Fig. 2.1 muestra el esquema de una secuencia (por lo que se denominan estructuras secuenciales), que en algunos lenguajes de programación se suele encerrar entre las palabras claves `begin` y `end` para indicar el comienzo y fin de la misma.

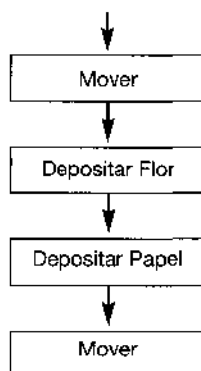


Figura 2.1

Por ejemplo, un recorrido en el cual el robot del Visual Da Vinci comienza desde la posición (1,1) con flores y papeles en su bolsa, da un paso, deposita una flor, deposita un papel, da otro paso y luego se detiene, se puede realizar mediante una secuencia como la siguiente:

Ejemplo 2.1

Precondiciones

- El robot está ubicado en la esquina (1,1)
- En su bolsa tiene flores y papeles

Poscondiciones

- El robot se encuentra en la esquina (1,3)
- En su bolsa hay una flor y un papel menos

```

{ El robot está ubicado en (1,1) y en su }
{ bolsa tiene flores y papeles          }

programa Uno
comenzar
    iniciar
    mover
    depositarFlor
    depositarPapel
    mover
fin

{ El robot se encuentra en (1,3) y      }
{ en su bolsa hay una flor y un papel menos }
```

2.1.1.2 Repetición

Una extensión natural de la secuencia consiste en repetir n veces un bloque de acciones.

Se considera que el número de veces que se deben ejecutar las acciones es fijo y conocido de antemano.

La Fig. 2.2 muestra el diagrama esquemático de la repetición.

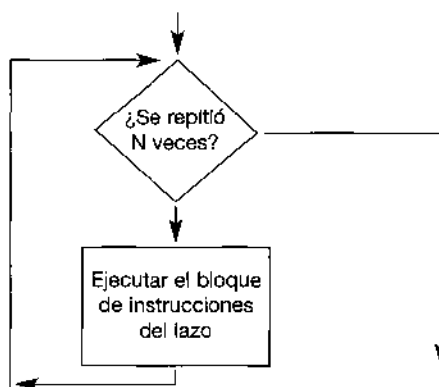


Figura 2.2

En el ejemplo anterior, si el robot debe repetir 10 veces las acciones de: avanzar, depositar una flor y depositar un papel (suponiendo que tiene suficientes papeles y flores en su bolsa), a partir de (1,1) resultaría el código del ejemplo 2.2:

	Ejemplo 2.2
	Precondiciones • El robot tiene al menos 10 flores y 10 papeles en su bolsa • Está ubicado en (1,1)
	Poscondiciones • En la bolsa ahora tiene 10 flores y 10 papeles menos • Está ubicado en la esquina (1,11)

```

{ El robot tiene al menos 10 flores y 10 papeles en sus }
{ bolsas y está ubicado en (1,1)                        }

programa Dos
comenzar
  iniciar
  repetir 10
    mover
    depositarFlor
    depositarPapel
  fin

{ En las bolsas ahora tiene 10 flores y 10 papeles menos, }
{ y está ubicado en (1, 11)                               }
    
```

Para esta estructura la sintaxis comúnmente utilizada por los lenguajes de programación es la siguiente:

	Código
	<pre> For indice := valor_inicial to valor_final do Acciones End do </pre>

En el ejemplo 2.3 se muestra su implementación, considerando que la variable A es de tipo entera.

Ejemplo 2.3

```
{ Contador es una variable de tipo INTEGER }  
  
A = 100;  
for Contador := 1 to 10 do  
  A := A + 2;  
  Imprimir(A);  
{ A contiene el valor 120 }
```

Algunas consideraciones generales de la estructura de control For son:

- For, to, do y End do son palabras clave del lenguaje. Hay que definir contador como una variable que se incrementa en una cantidad constante.
- Índice es una variable de tipo ordinal (tal como contador en el ejemplo anterior) que permite controlar el número de ejecuciones del ciclo.
- Valor_Inicial y Valor_Final son valores que indican el o los límites entre los que varía índice al comienzo y final del ciclo. Se debe notar que $\text{Valor_Final} - \text{Valor_Inicial} + 1$ indica el número de veces que se ejecuta el ciclo.
- Está implícito que en cada ciclo la variable índice toma el valor siguiente de acuerdo al tipo ordinal asociado a dicha variable. En el ejemplo, el **incremento es 1**. Se verá que en los diferentes lenguajes de programación se puede modificar este incremento o hacerlo negativo. De todos modos, conceptualmente la variable índice se actualiza en cada ciclo y se testea respecto al valor final deseado.
- Dentro de las Acciones del ciclo **no es posible modificar** la variable índice. Al terminar el ciclo, la variable índice no tiene un valor definido (su utilidad se limita a la estructura de repetición).

Otros ejemplos:

Ejemplo 2.4

```
{ Este bloque se ejecuta 8 veces }  
{ Se supone Índice como una variable CHAR }  
  
for Índice := 'A' to 'H' do  
begin  
  Acciones;  
end;
```

Ejemplo 2.5

```

{ Este bloque se ejecuta 2 veces }
{ Se supone Indice como una variable BOOLEAN }
for Indice := FALSE to TRUE do
begin
    Acciones;
end;
    
```

Ejemplo 2.6

```

{ Este bloque NO se ejecuta }
{ Se supone Indice como una variable INTEGER }
for Indice := 20 to 18 do
begin
    Acciones;
end;
    
```

2.1.1.3 Decisión

En un algoritmo representativo de un problema real es prácticamente imposible que las instrucciones sean secuenciales puras. Es necesario tomar decisiones en función de los datos del problema.

La estructura básica de decisión entre dos alternativas es la que se representa simbólicamente en la Fig.2.3.

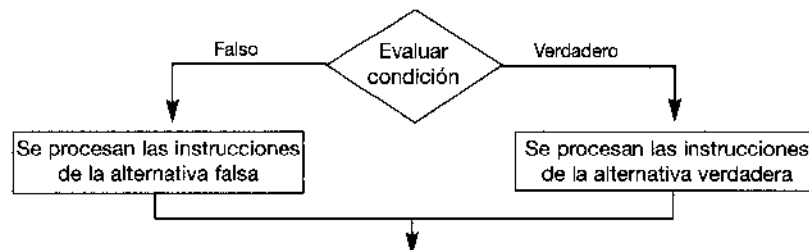


Figura 2.3

La sintaxis común para esta estructura en los lenguajes de programación es la siguiente:

Código

```

If (condición) then
    Acciones_por_Condición_Verdadera ;
Else
    Acciones_por_Condición_Falsa ;
End if
    
```

- If, then, else y End if son palabras clave del lenguaje.

- Condición es una expresión que al evaluarse devuelve un valor lógico TRUE o FALSE para tomar la decisión.

Por ejemplo:

Se tienen 3 variables enteras A, B y Min y se desea asignar en la variable Min el menor de los valores entre A y B. En el Ejemplo 2.7 se puede ver su implementación.



Ejemplo 2.7

```

{ A, B, Min son variables INTEGER      }
if A <= B then
    Min := A
else
    Min := B;
{ Min tiene el menor valor entre A y B }
```

Utilizando el ejemplo 2.2, supóngase ahora que el robot debe recoger un papel en cada calle impar a partir de (1,3); en caso de no poder hacerlo debe depositar otra flor. El segmento de algoritmo del Ejemplo 2.8 presenta los cambios necesarios.



Ejemplo 2.8

Precondiciones

- El robot está en la esquina (1,1)
- El robot cuenta con la cantidad de flores necesarias para cumplir el ciclo.

Poscondición

- Todas las esquinas con calles pares tienen flores y papeles.

```

{ El robot está en (1,1). Cuenta con la cantidad de flores      }
{ necesarias para cumplir el ciclo                               }
programa Tres
comenzar
    iniciar
    repetir 10
        mover
        depositarFlor
        depositarPapel
        mover
        si HayPapelEnLaEsquina
            tomarPapel
        sino
            depositarFlor
    fin
{ Todas las esquinas con calles pares tiene flores y papeles }
```

Puede ocurrir que no se tengan que representar acciones cuando la condición es falsa. Entonces la estructura quedaría reducida a:

	Código
<pre>IF (condición) Then Acciones_por_Condición_Verdadera End If</pre>	

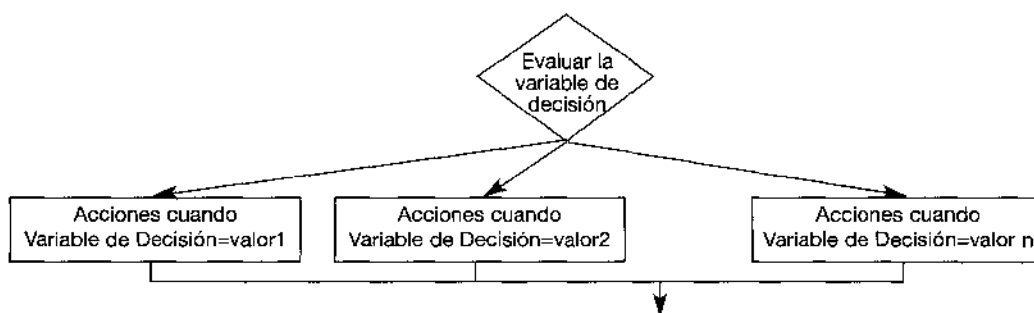
Utilizando el ejemplo 2.7, el algoritmo puede reescribirse del siguiente modo:

	Ejemplo 2.9
<pre>{ A, B, Min variables INTEGER } Min := B; if A <= B then Min := A; { Min tiene el menor valor entre A y B }</pre>	

Los lenguajes de programación pueden tener variantes que enriquezcan la estructura de decisión que se ha presentado (por ejemplo en ADA se tiene una variante de la estructura de control el IF OR ELSE IF e IF AND THEN IF) pero, normalmente, todos los lenguajes disponen de la estructura originalmente definida.

2.1.1.4 Selección

La Fig. 2.4 muestra una extensión de la estructura básica de decisión, para el caso de que las alternativas sean más de dos:



 **Figura 2.4**

La sintaxis común en los lenguajes de programación es:

Código
<pre> Select Case (Variable_Decisión) Case (posibilidad 1) Acciones para posibilidad 1 Verdadera ; Case (posibilidad 2) Acciones para posibilidad 2 Verdadera ; Case (posibilidad n) Acciones para posibilidad n Verdadera ; Else Acciones para todas las otras posibilidades no contempladas arriba; End Select </pre>

Algunas consideraciones generales de la estructura de control Case son:

- Select Case, Case, Else y End Select son palabras clave del lenguaje.
- Posibilidad n es una expresión que involucra a la Variable_Decisión. Al evaluarse devuelve un valor lógico, si es TRUE se ejecutan las acciones para esa posibilidad.
- Algunos lenguajes imponen que alguna de las n condiciones debe ser verdadera. En estos casos no existe cláusula Else.
- Los conjuntos de valores permitidos para cada posibilidad son disjuntos.

Ejemplo 2.10
El siguiente segmento de algoritmo permite leer las edades de personas entre 1 y 100 años. Se desea agrupar a las edades de las personas en 4 categorías: menores de 10 años, entre 10 y 25 años, entre 26 y 45 años y entre 46 y 100 años. Precondición • Se analiza un grupo de personas cuyas edades oscilan entre 1 y 100 años. Poscondición • En cada uno de los casos se tiene la cantidad total de personas que pertenecen a los intervalos correspondientes <pre> { edad es una variable entera positiva} { clase1, clase2, clase3 y clase4 con contadores enteros} { dato_invalido es un contador para edades fuera de rango} { inicializar contadores en 0 { leer la variable edad} </pre>

```

Select Case (edad)
    Case 1..9
        Clase1 :=clase1+1;
    Case 10..25
        Clase2 :=clase2+1;
    Case 26..45
        Clase3 :=clase3+1;
    Case 46..100
        Clase4 :=clase4+1;
    Else
        Dato_invalido :=dato_invalido + 1;
End Select
    
```

Ejemplo 2.11

Se supone que el robot está ubicado en una esquina cualquiera, y debe resolver qué dirección debe tomar de acuerdo a la cantidad de flores que tenga en la bolsa. Para ello debe averiguar, si hay 5 flores debe girar a la derecha, si hay 3 flores debe girar a la izquierda, si hay solo una flor debe seguir en la misma dirección, y en caso contrario debe quedarse en el mismo lugar.

Precondiciones

- El robot está ubicado en (1,5)
- De acuerdo a la cantidad de flores puede tomar una de las acciones.

Poscondición

- El robot queda orientado como corresponde

{ El robot está ubicado en (1,5) }

```

programa Cuatro
variables
    Flor: numero
comenzar
    iniciar
    pos (1,5)
    Flor := 0
    mientras HayFlorEnLaBolsa
        depositarFlor
        Flor := Flor + 1
    caso (Flor)
        5: Derecha

        3: repetir 3
            derecha

        1: Mover
    fin
        
```

Nota: En la versión 1.0 del ambiente Visual Da Vinci la estructura caso no está implementada.

2.1.1.5 Iteración

Puede ocurrir que se desee ejecutar un bloque de instrucciones desconociendo el número exacto de veces que se ejecutan. Para estos casos en la mayoría de los lenguajes de programación estructurada existen **estructuras de control iterativas condicionales**. Como su nombre lo indica, las acciones se ejecutan dependiendo de la evaluación de la condición definitiva.

Estas estructuras se clasifican en **pre-condicionales** y **pos-condicionales**. Las pre-condicionales evalúan la condición y, si es verdadera, se ejecuta el bloque de acciones; esto hace que dicho bloque se pueda ejecutar 0, 1 o más veces.

La Fig.2.5 muestra el esquema correspondiente:

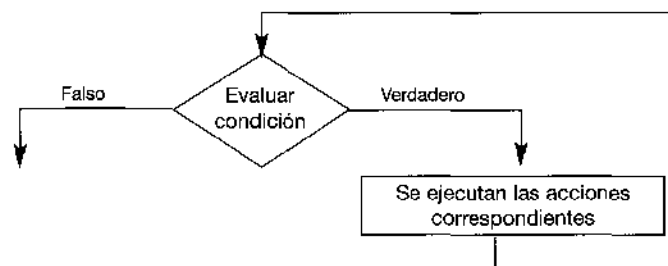


Figura 2.5

Ejemplo 2.12

Se desea resolver un algoritmo que lea una sucesión de números enteros, la cual finaliza al ingresar el valor 0. Se desea obtener la suma de los números leídos.

Precondición

- Se lee una secuencia de números que finaliza en 0.

Poscondición

- Se obtiene la suma total de los números.

```

{ En la variable Suma se guarda el resultado de la      }
{ suma de los números ingresados.                      }
{ Se debe inicializar la variable Suma                  }
}

Suma := 0;
WriteLn('Ingrese un número entero: ');
{ La variable Numero toma el valor que luego se utiliza para evaluar
  la condición por primera vez }
Read(Numero);
while Numero <> 0 do

```

```
begin
    Suma := Suma + Numero;
    WriteLn('Ingrese un número entero: ');
    { La variable Numero toma otro valor que utilizará como nueva
      condición }
    Read(Numero);
end;
WriteLn('El resultado de la suma es: ', Suma);
```

Ejemplo 2.13

Supóngase que el robot debe recoger todos los papeles de la esquina donde se encuentre ubicado, la solución podría escribirse como sigue:

Precondición

- El robot se encuentra ubicado en la esquina (10,10)

Poscondición

- No hay papeles en la esquina (10, 10)

{ El robot se ubica en la esquina (10,10) }

```
programa Cinco
comenzar
    iniciar
    Pos(10, 10)
    mientras HayPapelEnLaEsquina
        tomarPapel
    fin
```

{ No hay papeles en la esquina (10,10) }

Este ciclo termina cuando en la esquina no hay más papeles

En cuanto a las estructuras poscondicionales, primero se ejecuta el bloque de acciones, luego se evalúa la condición y, si es falsa, se ejecuta nuevamente el bloque de acciones. A diferencia de la estructura iterativa anterior, el bloque de acciones se debe ejecutar 1 o más veces. Nótese que, en este caso, el bloque de acción se ejecuta antes de evaluar la condición, por lo tanto se lleva a cabo al menos una vez. La Fig 2.6 muestra el esquema correspondiente.

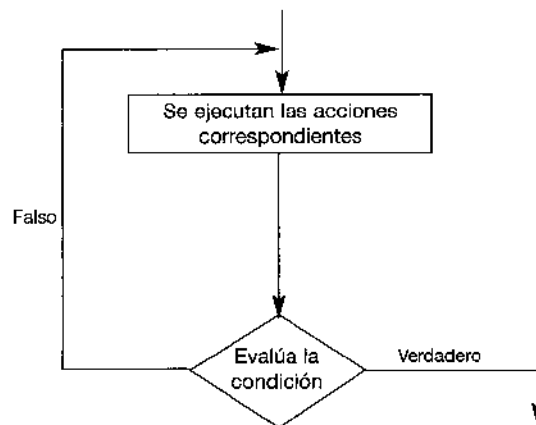


Figura 2.6

	Ejemplo 2.14
Supóngase que se desea procesar una secuencia de números para obtener su suma. Dicha secuencia termina cuando se ingresa el número 23, el que también debe ser procesado.	
Precondición • El primer número es distinto de 0	
Poscondición • Se obtiene la suma de los números ingresados	
<pre> { En la variable Suma se guarda el resultado de la } { suma de los números ingresados } { Se debe inicializar la variable Suma en 0 } } Suma := 0; repeat WriteLn('Ingrese un número entero: '); Read(Numero); Suma := Suma + Numero; until Numero = 0; WriteLn('El resultado de la suma es: ', Suma); { En Suma se acumula la suma de los números leídos } </pre>	



Ejemplo 2.15

Si el robot debe recorrer la avenida 1 hasta encontrar una esquina que tiene exactamente 5 flores, depositando todo el contenido de la bolsa en la esquina siguiente del recorrido, se podría escribir la siguiente solución:

Precondiciones

- El robot está ubicado en la esquina (1,1).
- En la avenida 1 existe una esquina con 5 flores.

Poscondición

- El robot encontró una esquina con 5 flores.

```

{ El robot está ubicado en la (1,1). }
{ La esquina seguro existe y es anterior a la calle 100 }

programa Seis
variables
  Flores: numero
comenzar
  iniciar
  mientras Flores <> 5
    Flores := 0
    mientras HayFlorEnLaEsquina
      Flores := Flores + 1
      tomarFlor
    mover
  mientras HayFlorEnLaBolsa
    depositarFlor
fin
{ El robot encontró una esquina con 5 flores }

```

2.1.2 Estructura esquemática de un programa

A continuación se presenta un caso que plantea la estructura esquemática de un programa, siguiendo para ello la sintaxis del lenguaje Pascal.



Código

```

Program Identificador;
{ sección de declaración de datos}
  Var id1: tipo;
      id2: tipo;
      .....
  { sección de sentencias ejecutables}
  begin
      .....
  end.

```



Donde Program, Var, Begin y End son palabras claves:

- Program indica el comienzo de un programa.
- Identificador es el nombre que se le dio al programa.
- Var indica la sección de declaración de los datos variables a utilizar en el programa. Esta sección se extiende hasta la palabra begin.
- Begin indica el comienzo de la sección de sentencias ejecutables del programa. Esta sección se extiende hasta la palabra end.
- End. indica el fin del programa.
- Los comentarios se escriben entre llaves en cualquier parte del programa.



Ejemplo 2.16

El siguiente algoritmo permite calcular la superficie de un terreno a partir de sus dimensiones.

Precondición

- Las longitudes del terreno son distintas de 0

Poscondición

- La variable superficie contiene el valor del área del terreno

```
{ Este programa calcula la superficie de un terreno }
{ Las longitudes del terreno son distintas de 0      }

program Superficie;
{ Sección de declaración de variables }
var
  Lado1: integer;      { longitud de un lado del terreno  }
  Lado2: integer;      { longitud de otro lado del terreno }
  Superficie: integer; { representa el área del terreno   }
begin
  { Sección de instrucciones ejecutables }
  WriteLn('Ingrese la longitud de un lado: ');
  ReadLn(Lado1);
  WriteLn('Ingrese la longitud de otro lado: ');
  ReadLn(Lado2);
  { Cálculo de la superficie }
  Superficie := Lado1 * Lado2;
  { Salida del resultado }
  WriteLn('La superficie del terreno es: ', Superficie);
end.
```

■ 2.2 Importancia de la documentación de un algoritmo

Un programa bien documentado será más fácil de leer y mantener. Para ello la documentación es fundamental. Es común que la mayoría de los lenguajes de programación provean algún mecanismo de documentación, por ejemplo, a través de la inserción de comentarios en el programa. Un programa sin comentarios revela un estilo de programación pobre y peligroso, ya que dificulta el mantenimiento adecuado del mismo.

Es recomendable utilizar un comentario general para el objetivo del programa o del módulo de programa en cuestión, que refleje la especificación del problema a resolver, de la forma más completa posible. Su lectura debe ser suficiente para poder entender las acciones que se llevan a cabo en el mismo. Además, deberán describirse todos los datos variables y constantes que intervengan en el programa, e indicar la fecha de realización del mismo, el autor, etc.

Por otra parte, se deberán elegir identificadores de manera tal que sean autoexplicativos. En cuanto a los comentarios intercalados en el programa (documentación on-line o interna), deben realizarse con cierto criterio, pues deben contribuir a la claridad del programa; por ejemplo, es importante comentar el estado inicial y final entre las acciones de una determinada secuencia, los efectos colaterales probables que puedan tener lugar luego de una llamada a un módulo, etc.

Es importante destacar que cuando se realiza el mantenimiento de un programa no solo se actualiza el código, sino también los comentarios del mismo. Debe quedar claro que los comentarios no son un agregado al programa, sino una parte importante del mismo.

■ 2.3 Corrección de algoritmos. Importancia de la verificación

Un programa es correcto cuando cumple con su especificación, esto significa que cumple con los requerimientos propuestos. Para determinar cuáles son esos requerimientos se debe tener una especificación completa, precisa y libre de ambigüedades del problema a resolver antes de escribir el mismo.

Una de las formas para saber si un programa es correcto consiste en probar con un juego de datos reales que permitan determinar que cumple con su especificación. Debido a la importancia de la verificación de los programas las técnicas empleadas evolucionan en búsqueda de mayor eficiencia. Este campo de investigación se actualiza constantemente en la Ciencia de la Computación.



Ejemplo 2.17

Supóngase que el robot debe vaciar su bolsa de flores depositando una flor en cada una de las esquinas de la ciudad. A continuación se presenta la solución a dicho problema.

Precondición

- El robot está ubicado en la esquina (1,1)

{ El robot está ubicado en la esquina (1,1) }

```
programa Siete
comenzar
  iniciar
  mientras HayFlorEnLaBolsa
    depositarFlor
  mover
fin
```

¿Pero esta solución es siempre correcta? La respuesta es **no**.

Y una de las razones surge ante la pregunta: ¿qué ocurre si en la bolsa hay más de 100 flores?

Una solución correcta, utilizando las mismas precondiciones, sería:



Ejemplo 2.18

```
programa Ocho
variables
  Pasos: numero
comenzar
  iniciar
  mientras HayFlorEnLaBolsa
    Pasos := 0
    mientras HayFlorEnLaBolsa & (Pasos < 99)
      depositarFlor
    mover
    Pasos := Pasos + 1
  si Pasos = 99
    derecha
    mover
    derecha
  Informar(PosAv, PosCa)
fin
```

2.4 Eficiencia de un algoritmo

A lo largo del texto se verá que se pueden tener varias soluciones algorítmicas para un mismo problema (por ejemplo, para encontrar el menor valor de una lista de números o para que el robot vaya de un punto a otro de la ciudad). Sin embargo, el uso de recursos (tiempo, memoria) para cada una de esas soluciones puede ser muy diferente.

Definición

Se define a la eficiencia como una métrica de calidad de los algoritmos, asociada con una utilización óptima de los recursos del sistema de cómputo donde se ejecutará el algoritmo.

Ejemplo 2.19

Supóngase que el robot debe ir desde la esquina (1,1) a la esquina (21,1). Un recorrido posible consiste en recorrer la calle 1 desde la avenida 1 a la avenida 21. Para este recorrido, la solución sería:

Precondición

- El robot está ubicado en la esquina (1,1)

Postcondiciones

- El robot está ubicado en la esquina (21,1)
- Caminó 20 cuadras

```
{ El robot está ubicado en la esquina (1,1) }
programa Nueve
comenzar
    iniciar
    derecha
    repetir 20
        mover
    fin
{ El robot está ubicado en la esquina (21,1) }
{ El robot caminó 20 cuadras }
```

Otro recorrido posible es el que se muestra en la Figura 2.7

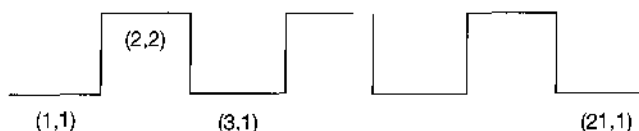


Figura 2.7

Ejemplo 2.20

Precondición

- El robot está ubicado en la esquina (1,1)

Poscondición

- El robot está ubicado en la esquina (21,1) y caminó 40 cuadras

{ El robot está ubicado en la esquina (1,1) }

```
programa Diez
comenzar
  iniciar
  derecha
  repetir 10
    mover
    repetir 3
      derecha
    repetir 2
      mover
      derecha
    mover
    repetir 3
      derecha
  fin
```

{ El robot está ubicado en la esquina (21,1) }

{ El robot caminó 40 cuadras }

Obsérvese que en ambas soluciones el robot llega a la esquina (21,1). En la primera solución camina 20 cuadras y en la segunda solución camina 40 cuadras.

2.5 Descomposición de problemas

2.5.1 Modularización

Hasta aquí se han analizado los algoritmos y los programas que dan solución a problemas simples y breves. Sin embargo, se podrá observar que, en general, los problemas del mundo real son más complejos y extensos. En algunas ocasiones, por ejemplo, una solución ya implementada deberá modificarse para satisfacer un nuevo requerimiento. Es por ello que se deben considerar algunas herramientas para facilitar la resolución de tales problemas. Algunas de estas herramientas tienen que ver con la abstracción y con la descomposición de los problemas a resolver. La abstracción permitirá representar los objetos relevantes en el dominio del problema,

y la descomposición es una técnica que se basa en el paradigma: "Divide y vencerás". La descomposición tiene un objetivo: dividir cada problema en subproblemas y estos, a su vez, en subproblemas más pequeños y así sucesivamente. Cada uno de los subproblemas finales resultará entonces más simple de resolver.

Cuando se realiza esta descomposición debe tenerse en cuenta que cada subproblema está en un mismo nivel de detalle, que cada uno se puede resolver independientemente de los demás y que las soluciones de estos subproblemas pueden combinarse para resolver el problema original.

El método de diseño descendente (*Top Down*) se lo conoce también como modularización y es importante para lograr un buen diseño de los programas. Esquemáticamente se la representa en la Figura 2.8

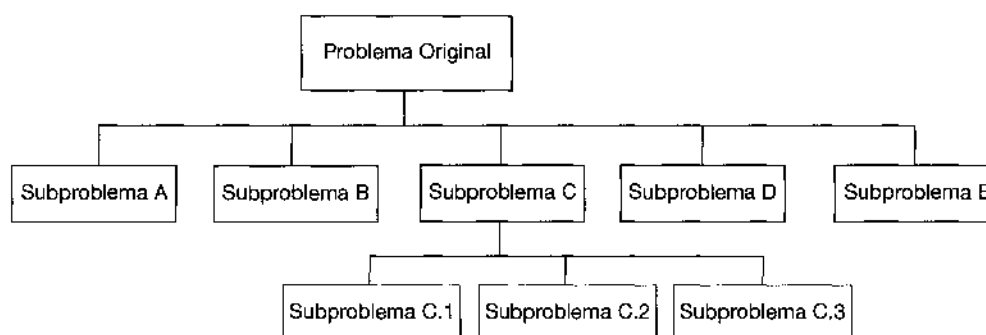


Figura 2.8

2.5.1.1 Importancia de la modularización

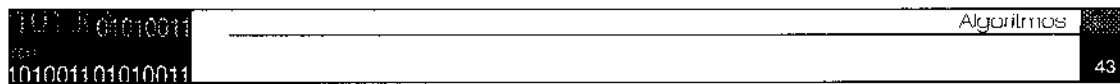
¿Por qué es importante una buena modularización?

El Conocimiento humano.

Algunas investigaciones acerca de la cognición han demostrado que existe un límite en cuanto a la cantidad de información que una persona puede tener en su mente a la vez, y que esa cantidad es de 7 ± 2 piezas de información. También demostraron que naturalmente la persona tiende a entender un problema de características complejas "partiendo" o "trozando" la información. Es decir, mientras la persona puede tener 7 u 8 piezas de información en mente a la vez, cada una de estas piezas, puede ser parte de otras 7 u 8 piezas distintas, estableciéndose de esta manera una relación jerárquica. En consecuencia, para comprender un problema complejo del mundo real es necesario dividir y subdividir, o sea: modularizar.

Trabajo en equipo.

La modularización favorece el trabajo en equipo ya que los productos de software complejos necesitan más de una persona para poder realizar el diseño, la codificación y la corrección de los mismos.



Por ejemplo, un programador normalmente trabaja desarrollando una parte del código (algunos módulos) desconociendo el resto del trabajo.

Como el programa esta modularizado, cada programador recibe las especificaciones de la tarea a realizar y las restricciones con las que debe manejarse.

Mantenimiento del programa.

Esta actividad involucra básicamente dos tareas: corregir los errores y modificar el código. A menudo, los errores pueden ocurrir antes de que el producto de software esté terminado o bien pueden ser descubiertos más tarde, es decir, cuando el producto ya se está utilizando. De todos modos, estos errores deben ser identificados, ya que se deben encontrar las causas y el código correspondiente debe ser corregido. Como es usual que se realicen cambios al software, por ejemplo, para el agregado de nuevas funcionalidades, esto también lleva a modificar el código existente.

La experiencia demuestra que el costo de mantenimiento en un código que no ha sido apropiadamente modularizado es muy alto. En cambio, el costo puede reducirse si el código está bien modularizado dado que el tiempo requerido para entender el programa y luego modificarlo es notoriamente menor.

Otro problema común cuando se realiza el mantenimiento de un programa es tener en cuenta los *side effects* (efectos colaterales), los que aparecen al cambiar el código para satisfacer cierta necesidad y producen efectos no deseados en otras partes del programa. Las causas de estos efectos y cómo prevenirlos se verán más adelante.

Importancia de la reusabilidad del código.

La idea básica es muy simple: cada vez que se crea un nuevo programa es deseable, de ser posible, hacer uso del código que ya se ha escrito. Esto es posible si se complementan soluciones bien modularizadas.

2.5.1.2 Módulos de un programa

Un buen diseño de programa significa, entre otras consideraciones, que un problema sea descompuesto en un número de subproblemas, que se denominarán a partir de aquí módulos. Cada módulo tendrá, entonces, una tarea específica bien definida y se comunicarán entre sí adecuadamente para conseguir un objetivo común.

Un módulo es simplemente un "trozo de código", o un conjunto de instrucciones más un conjunto de definiciones de datos que realiza una tarea lógica.

En el robot, cada uno de estos módulos de programa se denomina proceso.

Volviendo al ejemplo del robot: se desea que el robot realice recorridos en escalera, recogiendo todos los papeles que encuentre. La primera escalera comienza en la esquina (1,1) y fi-

naliza en la esquina (10,10). La segunda escalera comienza en la esquina (3,1) y finaliza en la esquina (10,8). La tercera escalera comienza en la esquina (5,1) y finaliza en la esquina (10,6), y así sucesivamente. En todos los casos cada escalón debe ser de una cuadra, debe informar para cada recorrido en escalera la cantidad de pasos dados, y por último la cantidad de papeles recogidos en todo el recorrido.

Se debe notar que existe un conjunto de soluciones, donde solo la última será realmente operativa en el ambiente Visual Da Vinci. Desde un punto de vista didáctico se trata de mejorar el esquema del programa teniendo en cuenta aspectos de la modularización, y luego de variables locales y parámetros, hasta llegar a la solución deseada.

El algoritmo resultante se presenta así:



Ejemplo 2.21
Precondición

- El robot está en (1,1) orientado hacia el norte

Poscondiciones

- El robot queda posicionado en (10,2), orientado hacia el norte
- Cuenta la cantidad total de papeles en todo el recorrido


```
{ El robot está en (1,1) orientado hacia el norte }  
  
programa Recorrido  
variables  
  Escalones: numero    { Indica la cantidad de escalones de cada  
                        recorrido }  
  Papeles: numero      { Indica la cantidad de papeles de cada  
                        recorrido }  
  TotalPapeles: numero { Indica la cantidad total de papeles de  
                        todos los recorridos }  
  Avenida: numero      { Indica la avenida en la que comienza el  
                        recorrido }  
  Pasos: numero        { Indica los pasos dados en cada recorrido }  
comenzar  
  iniciar  
  { Inicializar las variables (1)}  
  Escalones := 9  
  TotalPapeles := 0  
  Avenida := 1  
  { Hacer los 5 recorridos }  
  repetir 5  
    { Hacer una escalera contando la cantidad de papeles (2)}  
    Papeles := 0  
    Pasos := 0
```

```
repetir Escalones
  mientras HayPapelEnLaEsquina
    tomarPapel
    Papeles := Papeles + 1
  mover
  Pasos := Pasos + 1
  derecha
  mientras HayPapelEnLaEsquina
    tomarPapel
    Papeles := Papeles + 1
  mover
  Pasos := Pasos + 1
  repetir 3
    derecha
    { Juntar los papeles de la última esquina del recorrido }
  mientras HayPapelEnLaEsquina
    tomarPapel
    Papeles := Papeles + 1
  { Informar la cantidad de pasos encontrados }
  Informar(Pasos)
  { Incrementar total de papeles del recorrido (3)}
  TotalPapeles := TotalPapeles + Papeles
  { Reposicionar al robot y actualizar cantidad de escalones (4)}
  Avenida := Avenida + 2
  Pos(Avenida, 1)
  Escalones := Escalones + 2
  { Informar la cantidad total de papeles del recorrido completo (5)}
  Informar(TotalPapeles)
fin

{ El robot queda posicionado en (10,2), mirando hacia el norte y en }
{ TotalPapeles tiene la cantidad total de papeles de todo el recorrido }
```

Se puede observar que en este programa hay cinco módulos (numerados a continuación de los comentarios). El primero obtiene los datos iniciales para realizar el recorrido. Una vez obtenida esta información, el segundo realiza el recorrido en escalera, contando los papeles encontrados y la cantidad de pasos dados. Luego, se puede actualizar la cantidad total de papeles del recorrido. En el paso siguiente se posiciona al robot y se recalcula la cantidad de escalones para el nuevo recorrido y, por último, se informa la cantidad total de papeles encontrados.

Si bien es cierto que este programa resulta corto y simple de resolver de manera lineal, es aconsejable utilizar los beneficios de la modularización referenciados al principio del tema. Para reflejar una solución modularizada se proponen cinco procesos, uno para cada módulo de acuerdo a la Figura 2.9.

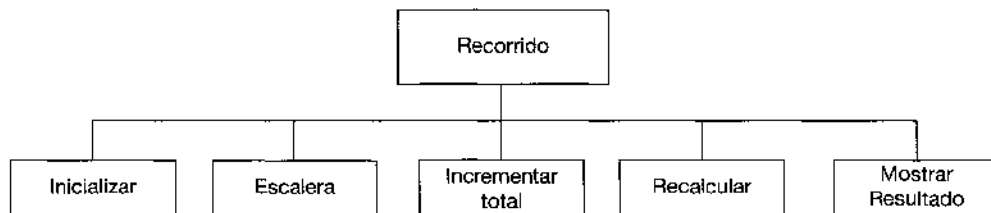


Figura 2.9

Se puede escribir, entonces, la siguiente solución:

Ejemplo 2.22

```

{ El robot está en (1,1) orientado hacia el norte }

programa Recorrido
variables
    Escalones: numero    { Indica la cantidad de escalones de cada
                           recorrido }
    Papeles: numero      { Indica la cantidad de papeles de cada
                           recorrido }
    TotalPapeles: numero { Indica la cantidad total de papeles de
                           todos los recorridos }
    Avenida: numero      { Indica la avenida en la que comienza el
                           recorrido }
    Pasos: numero        { Indica los pasos dados en cada recorrido }

procesos
    proceso Inicializar
        comenzar
            Escalones := 9
            TotalPapeles := 0
            Avenida := 1
        fin

    proceso Izquierda
        comenzar
            repetir 3
                derecha
            fin
    
```

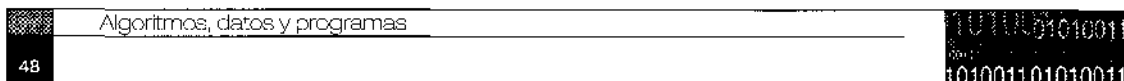
```
proceso Escalera
comenzar
  Papeles := 0
  Pasos := 0
  repetir Escalones
    mientras HayPapelEnLaEsquina
      tomarPapel
      Papeles := Papeles + 1
    mover
    Pasos := Pasos + 1
    derecha
    mientras HayPapelEnLaEsquina
      tomarPapel
      Papeles := Papeles + 1
    mover
    Pasos := Pasos + 1
    Izquierda
    { juntar los papeles de la última esquina del recorrido }
    mientras HayPapelEnLaEsquina
      tomarPapel
      Papeles := Papeles + 1
    { Informar la cantidad de pasos dados }
    Informar(Pasos)
  fin

proceso IncrementarTotal
comenzar
  TotalPapeles := TotalPapeles + Papeles
fin

proceso Recalcular
comenzar
  Avenida := Avenida + 2
  Pos(Avenida, 1)
  Escalones := Escalones - 2
fin

proceso MostrarResultado
comenzar
  Informar(TotalPapeles)
fin

comenzar { programa principal }
  iniciar
  Inicializar
  repetir 5
    Escalera
    IncrementarTotal
    Recalcular
    MostrarResultado
  fin
```



A continuación se analiza la ejecución de este programa.

Una vez declaradas las variables, las constantes y los procesos, la ejecución comienza con la primera instrucción del programa principal después de comenzar. Esta instrucción es *Iniciar* (posiciona al robot en (1,1)), continúa con una llamada al proceso *Inicializar*, la cual causa que la ejecución secuencial del programa principal quede temporariamente suspendida y sea ejecutado el código del proceso *Inicializar*. Cuando el proceso *Inicializar* termina, el control retorna al punto siguiente desde donde fue llamado y la ejecución secuencial continúa. Luego se encuentra la estructura *repetir*, que permite que se ejecute 5 veces el bloque de instrucciones que aparece a continuación. Se ejecuta, entonces, la llamada al proceso *Escalera*, la que produce que la ejecución secuencial se suspenda nuevamente y comiencen a ejecutarse las instrucciones de este proceso. Una vez que este proceso termina, el flujo de control retorna a la instrucción siguiente de la que se lo llamó desde el programa principal, y retoma nuevamente la ejecución secuencial. Continúa así hasta finalizar con la ejecución del último proceso que retorna el control al programa principal, para comenzar otra vez con este bloque de tres instrucciones (llamadas a procesos) hasta completar las 5 veces de la estructura *repetir*, luego se invoca *MostrarResultado* que procede de igual manera. Finalmente, ejecuta la instrucción siguiente *end* con lo cual termina la ejecución del programa.

Es importante destacar algunos aspectos de esta solución:

- Los procesos están declarados antes del comienzo del programa principal. Esto se debe a una regla general que dice que todos los procesos que utilice un programa, deben ser conocidos antes de ser usados.
- Si se compara la última solución con la anterior se puede observar que no hay código nuevo, excepto las llamadas a los procesos.
- El programa principal queda como un resumen de alto nivel del programa total. Como resultado final, el programa principal representa una especificación del programa. Este programa principal es simple y corto ya que la tarea de detalle queda oculta (*hidden*) en los módulos del programa.

2.5.2 Alcance de los datos

La importancia del ocultamiento de los datos (*Data Hidding*)

En el desarrollo de un sistema complejo con gran número de módulos, realizados por diferentes programadores, debe tenerse en cuenta que cada uno de ellos escribirá el código correspondiente a un módulo. Esto implica que cada cual define sus propios datos, con identificadores apropiados y como consecuencia de ello se podrían observar algunos inconvenientes tales como:

- Demasiados identificadores.

- Conflictos entre los nombres de los identificadores de cada programador.
- Integridad de los datos lo que implica que se puedan utilizar datos que tengan igual identificador pero que realicen funciones diferentes.
- *Side Effects* no previstos.

La solución para reducir estos problemas se logra con una combinación de ocultamiento de datos (*Data Hidding*) y uso de "Parámetros". Estos conceptos se discutirán a continuación.

2.5.2.1 Datos locales y datos globales

La diferencia entre dato global y dato local es simple: los datos globales son aquellos que se declaran en la sección de declaración del programa principal. En el ejemplo 2.21, todos los datos son globales.

Los datos locales son aquellos que se declaran en la sección de declaración de un módulo individual.

2.5.2.2 Ocultamiento y protección de datos ("*Data Hidding*")

El concepto de *Data Hidding* se utiliza aquí para significar que todo dato que es relevante para un módulo debe ocultarse de los otros módulos. De esta manera se evita que en el programa principal se declaren datos que sólo son relevantes para algún módulo en particular y, además, se protege la integridad de los datos.



Ejemplo 2.23

Si se vuelve al ejemplo planteado anteriormente, se observa que algunos datos pueden ser declarados en forma local y otros seguirán siendo globales.
En el proceso Escalera se puede observar que la variable Pasos puede ser declarada local al módulo, ya que solo es usada en dicho módulo.
a continuación se reescribe el ejemplo planteado, utilizando variables locales y globales.

```
{ El robot está en (1,1) orientado hacia el norte }  
  
programa Recorrido  
  
variables  
  Escalones: numero    { Indica la cantidad de escalones de cada  
                        recorrido }  
  Papeles: numero      { Indica la cantidad de papeles de cada  
                        recorrido }  
  TotalPapeles: numero { Indica la cantidad total de papeles de  
                        todos los recorridos }
```

```
Avenida: numero { Indica la avenida en la que comienza el recorrido }

procesos
  proceso Inicializar
  comenzar
    Escalones := 9
    TotalPapeles := 0
    Avenida := 1
  fin

  proceso Izquierda
  comenzar
    repetir 3
      derecha
    fin

  proceso Escalera
  variables
    Pasos: numero { Indica los pasos dados en cada recorrido }
  comenzar
    Papeles := 0
    Pasos := 0
    repetir Escalones
      mientras HayPapelEnLaEsquina
        tomarPapel
        Papeles := Papeles + 1
      mover
      Pasos := Pasos + 1
      derecha
      mientras HayPapelEnLaEsquina
        tomarPapel
        Papeles := Papeles + 1
      mover
      Pasos := Pasos + 1
      Izquierda
      { juntar los papeles de la última esquina del recorrido }
      mientras HayPapelEnLaEsquina
        tomarPapel
        Papeles := Papeles + 1
      { Informar la cantidad de pasos dados }
      Informar(Pasos)
    fin

  proceso IncrementarTotal
  comenzar
    TotalPapeles := TotalPapeles + Papeles
  fin
```

```
proceso Recalcular
comenzar
  Avenida := Avenida + 2
  Pos(Avenida, 1)
  Escalones := Escalones - 2
fin

proceso MostrarResultado
comenzar
  Informar(TotalPapeles)
fin

comenzar { programa principal }
  iniciar
  Inicializar
  repetir 5
    Escalera
    IncrementarTotal
    Recalcular
    MostrarResultado
  fin
```

Si se observa el código escrito, se puede notar que no se ha realizado ningún cambio en los datos utilizados, solo se ha cambiado el lugar donde se declaran. Esto produce dos consecuencias lógicas: permite una mejor organización de los datos y logra que los mismos estén protegidos.

Cuando se hace referencia a una mejor organización de los datos, se quiere significar que solo se declaran datos globales en el caso que los mismos sean utilizados por más de un módulo. Para aquellos datos que solo son utilizados en un módulo particular se declaran locales a dicho módulo, de manera que permanecen ocultos (*hidden*) para los demás módulos alcanzando, de esta manera, la protección de tales datos y, en consecuencia, la integridad de los mismos.

2.5.2.3 Alcance de los datos

Hasta aquí se han utilizado los términos local y global de manera absoluta, pero se debe considerar que los módulos pueden anidarse. Así como se declaran módulos dentro del programa principal, pueden declararse módulos dentro de otros módulos. Por lo tanto, deben aplicarse dos reglas muy importantes que gobiernan el alcance de los identificadores, y que se mencionan a continuación:

- El alcance de un identificador es el bloque de programa donde se lo declara.
- Si un identificador declarado en un bloque es nuevamente declarado en un bloque interno al primero, el segundo bloque es excluido del alcance de la primera sección, (véase figura 2.10).

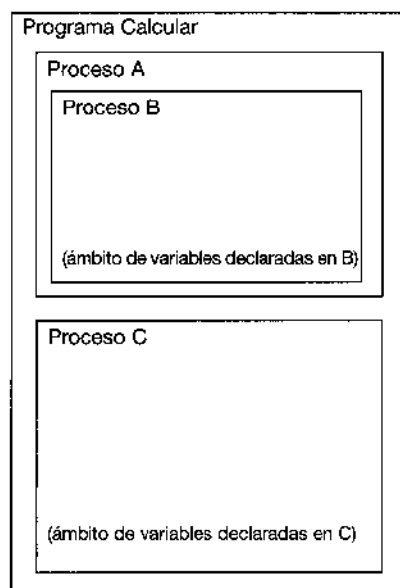


Figura 2.10

2.5.2.4 Parámetros

Los parámetros son variables que tienen como característica principal transferir información entre los módulos.

Cuando un programa está bien organizado e implementado utiliza parámetros para todos los datos compartidos entre módulos. Se puede adoptar, como regla general, que toda la información que viaja hacia o desde un módulo debe hacerse a través de parámetros, en lugar de utilizar variables globales a las que se accede desde todos los módulos.

Hay dos tipos de parámetros que interesan, los parámetros “pasados por valor” y los parámetros “pasados por referencia”.

Cuando existen datos compartidos entre módulos, una solución es que un módulo pase una copia de esos datos al otro. En este caso, el parámetro que se pasa se denomina parámetro “pasado por valor”. El módulo que recibe esta copia no puede efectuar ningún cambio sobre el dato original.

Los parámetros “pasados por referencia”, a diferencia de los parámetros “pasados por valor”, no envían una copia del valor del dato, sino que envían la dirección donde se encuentra el dato, con lo cual tanto el programa o el proceso que llama, como así también el proceso llamado, puede acceder a dicho dato.

A continuación se analizan algunas de las razones que permitirán explicar por qué es preferible el uso de parámetros al uso de variables globales, obviamente en aquellas situaciones en las que es posible elegir alguna de las dos alternativas:

Integridad de los datos

En este caso es necesario conocer qué datos usa cada módulo. Este problema puede solucionarse, en parte, declarando tales datos como locales al módulo, es decir, ocultándolos de los demás módulos, (esto se presenta en la solución planteada anteriormente). Sin embargo, se puede observar que el problema no queda resuelto cuando los datos deben ser compartidos por distintos módulos. La solución, entonces, implica conocer qué datos globales requiere cada uno de los módulos y qué datos globales son compartidos por los distintos módulos. Además, es importante saber si un módulo particular utiliza el dato solo de "lectura" o bien lo modifica. Es aquí, donde se utilizan los parámetros.

Uso de parámetros para proteger los datos

Si una variable es local a un módulo se puede asegurar que cualquier otro módulo fuera del alcance de esa variable no la puede "ver" y, por lo tanto, no la puede modificar. Dado que las variables globales pueden ser accedidas desde distintas partes del programa, no se puede garantizar que accidentalmente no se modifiquen. La solución al problema será utilizar parámetros "pasados por valor".

Volviendo al proceso Escalera, es necesario reconocer la cantidad de escalones para cada uno de los recorridos. Se puede, entonces, reescribir la solución de la siguiente manera:



Ejemplo 2.24

```
{ El robot está en (1,1) orientado hacia el norte }  
programa Recorrido  
variables  
  Escalones: numero    { Indica la cantidad de escalones de cada  
                        recorrido}  
  Papeles: numero      { Indica la cantidad de papeles de cada  
                        recorrido}  
  TotalPapeles: numero { Indica la cantidad total de papeles de  
                        todos los recorridos}  
  Avenida: numero      { Indica la avenida en la que comienza el  
                        recorrido}  
  
procesos  
  proceso Inicializar  
  comenzar  
    Escalones := 9  
    TotalPapeles := 0  
    Avenida := 1  
  fin
```

```
proceso Izquierda
comenzar
  repetir 3
    derecha
  fin

proceso Escalera (E Esc: numero)
variables
  Pasos: numero           { Indica los pasos dados en cada
                           recorrido }
comenzar
  Papeles := 0
  Pasos := 0
  repetir Esc
    mientras HayPapelEnLaEsquina
      tomarPapel
      Papeles := Papeles + 1
    mover
    Pasos := Pasos + 1
    derecha
    mientras HayPapelEnLaEsquina
      tomarPapel
      Papeles := Papeles + 1
    mover
    Pasos := Pasos + 1
    Izquierda           { juntar los papeles de la última
                           esquina del recorrido}
  mientras HayPapelEnLaEsquina
    tomarPapel
    Papeles := Papeles + 1 { Informar la cantidad de pasos dados }
  Informar(Pasos)
fin

proceso IncrementarTotal(E Pap: numero)
comenzar
  TotalPapeles := TotalPapeles + Pap
fin

proceso Recalcular
comenzar
  Avenida := Avenida + 2
  Pos(Avenida, 1)
  Escalones := Escalones - 2
fin

proceso MostrarResultado(E TotPap: numero)
comenzar
  Informar(TotPap)
fin
```

```
{programa principal}
comenzar
  iniciar
  Inicializar
  repetir 5
    Escalera(Escalones)
    IncrementarTotal(Papeles)
    Recalcular
  MostrarResultado(TotalPapeles)
fin
```

Se puede observar que en esta solución se ha cambiado el encabezamiento (*Header*) del proceso Escalera y la llamada a dicho proceso. Al cambiar el encabezamiento, este proceso espera recibir como dato de entrada la variable Escalón. Este dato le permitirá ejecutar el recorrido esperado. Por otro lado, en la llamada al proceso se especifica el dato que debe enviarse al proceso como parámetro. El número de parámetros enviados (en la invocación) y de parámetros recibidos (en el *header*) deben coincidir. Los parámetros enviados se denominan parámetros “actuales” y los parámetros que aparecen en el encabezamiento del proceso se definen como parámetros “formales”. La asociación entre los parámetros actuales y los parámetros formales se hace por su posición en la lista respectiva de parámetros, así el primer parámetro enviado en la lista de parámetros actuales se asocia al primer parámetro de la lista de parámetros formales, y así sucesivamente.

Por ejemplo, en el lenguaje Pascal, debido a su característica de “fuertemente tipado”, además de coincidir en el número de parámetros actuales y formales, deben coincidir en el tipo de dato asociado a cada uno de los parámetros actuales y formales respectivamente ya que, en caso contrario, se producen errores en tiempo de compilación. Estos conceptos se analizarán con más detalles en el capítulo 4.

Uso de parámetros para retornar datos

El parámetro “pasado por valor” es útil para proteger los datos. El enviar una copia de la información evita que el módulo lo modifique. Sin embargo, existen situaciones en las que se quiere conocer qué modificaciones se hicieron sobre los datos. Para retornar datos al módulo que hizo el llamado se utilizan los parámetros “pasados por referencia”. Cuando se utilizan parámetros en un módulo se deben diferenciar cuáles son “por valor” y cuáles “por referencia” ya que los datos se pasan de manera diferente. Para distinguir los parámetros “pasados por referencia”, por ejemplo, en Pascal, se le antepone la palabra *Var*.



Ejemplo 2.25
Por ejemplo, el proceso Escalera debe devolver la cantidad de papeles encontrados en el recorrido, el proceso Inicializar debe retornar los valores iniciales para comenzar el recorrido. Se puede, entonces, reescribir la solución de la siguiente manera:

```
{ El robot está en (1,1) orientado hacia el norte }
programa Recorrido
variables
  Escalones: numero    { Indica la cantidad de escalones de cada
                        recorrido}
  Papeles: numero      { Indica la cantidad de papeles de cada
                        recorrido}
  TotalPapeles: numero { Indica la cantidad total de papeles de
                        todos los recorridos}
  Avenida: numero      { Indica la avenida en la que comienza el
                        recorrido}

procesos
  proceso Inicializar(S Esc: numero; S TotPap: numero; S Ave: numero)
  comenzar
    Esc := 9
    TotPap := 0
    Ave := 1
  fin

  proceso Izquierda
  comenzar
    repetir 3
      derecha
    fin

  proceso Escalera (E Esc: numero; S Pap: numero)
  variables
    Pasos: numero      { Indica los pasos dados en cada recorrido }
    PapAux: numero
  comenzar
    PapAux := 0
    Pasos := 0
    repetir Esc
      mientras HayPapelEnLaEsquina
        tomarPapel
        PapAux := PapAux + 1
      mover
        Pasos := Pasos + 1
      derecha
      mientras HayPapelEnLaEsquina
        tomarPapel
        PapAux := PapAux + 1
      mover
        Pasos := Pasos + 1
      Izquierda      { juntar los papeles de la última esquina del
                     recorrido}
    mientras HayPapelEnLaEsquina
      tomarPapel
```

```
        PapAux := PapAux + 1
        Pap := PapAux { Informar la cantidad de pasos dados }
        Informar(Pasos)
    fin

proceso IncrementarTotal(E Pap: numero)
comenzar
    TotalPapeles := TotalPapeles + Pap
fin

proceso Recalcular
comenzar
    Avenida := Avenida + 2
    Pos(Avenida, 1)
    Escalones := Escalones - 2
fin

proceso MostrarResultado(E TotPap: numero)
comenzar
    Informar(TotPap)
fin

{programa principal}
comenzar
    iniciar
    Inicializar(Escalones, TotalPapeles, Avenida)
    repetir 5
        Escalera(Escalones, Papeles)
        IncrementarTotal(Papeles)
        Recalcular
        MostrarResultado(TotalPapeles)
    fin
```

Hasta aquí la solución presenta todos los datos de entrada como parámetros pasados por valor y los datos de salida como parámetros por referencia.

Uso de parámetros para enviar datos en ambos sentidos

Como se vio, los parámetros “pasados por valor” se utilizan como parámetros de entrada y los “pasados por referencia” como parámetros de salida. Existe una tercera posibilidad que es cuando un parámetro es usado como entrada y salida. Esto ocurre cuando, por ejemplo, un dato es enviado como entrada a un módulo, luego el dato es modificado en ese módulo y, por último, el dato modificado debe retornar al módulo que hizo el llamado. En estas circunstancias el parámetro debe ser declarado como parámetro “pasado por referencia”, ya que de otro modo queda protegido y no se conoce la modificación fuera del módulo. Como conclusión, se utilizan parámetros “pasados por referencia” en dos circunstancias: cuando se necesita que el parámetro sea de “salida” o bien, cuando se necesita que el parámetro sea de “entrada/salida”. No hay ninguna diferencia en cuanto al mecanismo empleado para estos dos casos, sin em-

bargo, es importante destacar, que la distinción debe hacerse explícita en la documentación del programa correspondiente.

Ejemplo 2.26

En el ejemplo se puede observar que en el proceso IncrementarTotal, la variable TotalPapeles es de entrada/salida, y en el proceso Recalcular, las variables Avenida y Escalones también son de entrada/salida.

Se puede, entonces, reescribir la solución considerando ahora en forma completa todos los datos de entrada, de salida y los datos de entrada/salida de la siguiente manera:

```
programa Recorrido
procesos
  proceso Inicializar(S Esc: numero; S TotPap: numero; S Ave: numero)
  comenzar
    Esc := 9
    TotPap := 0
    Ave := 1
  fin

  proceso Izquierda
  comenzar
    repetir 3
      derecha
    fin

  proceso Escalera (E Esc: numero; S Pap: numero)
  variables
    Pasos: numero { Indica los pasos dados en cada recorrido }
    PapAux: numero
  comenzar
    PapAux := 0
    Pasos := 0
    repetir Esc
      mientras HayPapelEnLaEsquina
        tomarPapel
        PapAux := PapAux + 1
      mover
        Pasos := Pasos + 1
      derecha
      mientras HayPapelEnLaEsquina
        tomarPapel
        PapAux := PapAux + 1
      mover
        Pasos := Pasos + 1
      Izquierda { juntar los papeles de la última esquina del
        recorrido}
      mientras HayPapelEnLaEsquina
```

```

    tomarPapel
    PapAux := PapAux + 1
    Pap := PapAux { Informar la cantidad de pasos dados }
    Informar(Pasos)
fin

proceso IncrementarTotal(E Pap: numero; ES TotPap: numero)
comenzar
    TotPap := TotPap + Pap
fin

proceso Recalcular(ES Ave: numero; ES Esc: numero)
comenzar
    Ave := Ave + 2
    Pos(Ave, 1)
    Esc := Esc + 2
fin

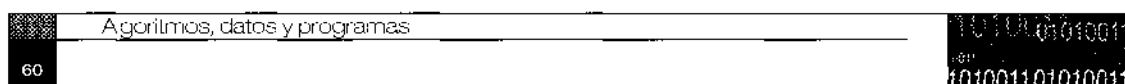
proceso MostrarResultado(E TotPap: numero)
comenzar
    Informar(TotPap)
fin

variables
    Escalones: numero { Indica la cantidad de escalones de cada
                        recorrido}
    Papeles: numero { Indica la cantidad de papeles de cada
                     recorrido}
    TotalPapeles: numero { Indica la cantidad total de papeles de
                           todos los recorridos}
    Avenida: numero { Indica la avenida en la que comienza el
                     recorrido}

{programa principal}
comenzar
    iniciar
    Inicializar(Escalones, TotalPapeles, Avenida)
    repetir 5
        Escalera(Escalones, Papeles)
        IncrementarTotal(Papeles, TotalPapeles)
        Recalcular(Avenida, Escalones)
        MostrarResultado(TotalPapeles)
    fin

```

Observación: el lector debe notar que la estructura esquemática presentada aquí varía de la presentada en Apéndice A. Esto se debe a detalles de implementación del robot.



Utilidad del uso de parámetros

Al escribir programas el uso de parámetros independiza a cada programador del nombre de los identificadores que usen los demás. Por otro lado, al probar un código es muy útil conocer la interfaz entre los módulos y generar pruebas asignando valores a cada uno de los parámetros que intercambian. Es sabido, también, que los programas requieren mantenimiento. Si estos programas se han diseñado en forma modular, la tarea de mantenimiento se realizará a menor costo. Esto es debido, en parte, a que la tarea de localizar errores será más fácil y se podrá determinar rápidamente cuáles son los módulos que se verán afectados por la corrección de esos errores.

Reuso de código

El uso de parámetros permite separar el nombre del dato, del dato en sí mismo, lo que permite que el mismo código sea utilizado en diferentes partes de un programa, simplemente cambiando la lista de parámetros actuales.

Dado que, a partir de aquí, los ejemplos de aplicación se escribirán en lenguaje Pascal, es importante aclarar que este lenguaje provee dos maneras de implementar módulos: **Procedure** (procedimiento) y **Function** (función).

Los procedimientos permiten devolver más de un dato al programa o módulo que lo invoca, y el flujo de control del programa retorna a la instrucción siguiente a la del llamado.

Las funciones permiten devolver un único valor al programa o módulo de programa que la invoca y el flujo de control del programa retorna a la misma instrucción que efectuó el llamado.

Más adelante, se estudiarán y presentarán cada uno de estos módulos con mayor detalle.

Conclusiones

Se pueden puntualizar algunas reglas básicas de trabajo:

1. Se debe adoptar un método de descomposición del problema *top down*, o sea: analizar el problema, dividirlo en subproblemas y reiterar el método con cada subprograma (módulo) hasta reducir la tarea a una función o procedimiento bien definido.
2. Se deben usar datos locales siempre que sea posible, y utilizar datos globales cuando no sea posible utilizar datos locales.
3. La comunicación entre módulos debe definirse a través de parámetros. Los módulos no deberían acceder a los datos globales, sino recibir los datos necesarios como parámetros.
4. Es imprescindible documentar cada uno de los módulos, haciendo algún comentario acerca del objetivo del módulo y precondiciones y poscondiciones para los datos utilizados.
5. Se debe notar que las poscondiciones indican el efecto sobre los datos del proceso. Se debe indicar cuál es el efecto del módulo.